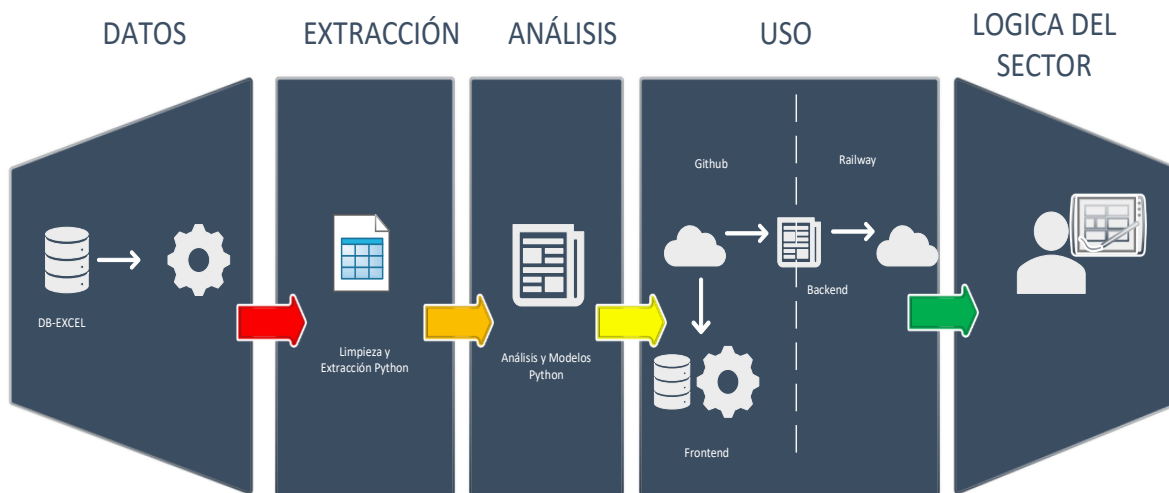


PROYECTO CONTUGAS GRUPO 20- ANEXO TÉCNICO

1. Diagrama esquemático propuesto:

A continuación, se describe el diagrama que representa la arquitectura general del sistema implementado, incluyendo los flujos de datos entre los componentes:



La figura presentada ilustra la arquitectura integral del sistema de monitoreo inteligente desarrollado para detectar anomalías en el consumo de gas de los clientes de Contugas. Este sistema se estructura en cinco fases secuenciales que garantizan la trazabilidad del flujo de datos desde su origen hasta la interfaz final de usuario:

1. Datos:

La base de datos inicial proviene de archivos Excel que simulan registros históricos de consumo, presión y temperatura. Estos datos representan la fuente primaria sobre la cual se construye el análisis.

2. Extracción:

Mediante scripts en Python, se realiza la limpieza, estructuración y extracción de las variables necesarias, incluyendo componentes temporales derivados de la fecha como mes, día de la semana y secuencia temporal.

3. Análisis:

En esta fase se aplican modelos de aprendizaje automático para la detección de anomalías. El enfoque final implementado consiste en una combinación de regresión

SVR para estimar el volumen esperado de consumo y el algoritmo DBSCAN para identificar puntos o clústeres de comportamiento atípico. Este modelo fue seleccionado por su capacidad de capturar patrones específicos de comportamiento y estacionalidad en los datos históricos de cada cliente.

4. **Uso:**

La implementación del sistema se despliega mediante una arquitectura cliente-servidor. El **frontend**, desarrollado en HTML/CSS/JavaScript, ofrece una visualización intuitiva de resultados y permite aplicar filtros interactivos. El **backend**, construido con Flask, expone una API RESTful que sirve los datos procesados, indicadores clave (KPIs), gráficos y resultados de clustering. Todo el sistema se aloja en Railway y su código fuente se gestiona mediante GitHub para facilitar el control de versiones y la colaboración.

5. **Lógica**

del

Sector:

Finalmente, los resultados se presentan en un entorno visual accesible para los usuarios del sector energético, quienes pueden interpretar fácilmente los registros de consumo normal y anómalo, facilitando la toma de decisiones técnicas o comerciales sobre posibles fugas, fallos en medidores o intervenciones externas.

Los componentes expuestos utilizaron las siguientes tecnologías:

Componente	Tecnología
Frontend	HTML, CSS, JavaScript
Backend (API)	Flask
Modelos de Análisis	SVR, DBSCAN
Almacenamiento Temporal	Archivos .xlsx
Visualización	Gráficos interactivos
Control de Versiones	GitHub
Despliegue	Railway

2. Reporte técnico de experimentos:

Esta sección describe de forma detallada los experimentos realizados para detectar patrones de consumo anómalos en clientes de Contugas, aplicando técnicas de clustering y regresión. Se evaluaron cuatro enfoques principales:

2.1. DBSCAN con 3 Features

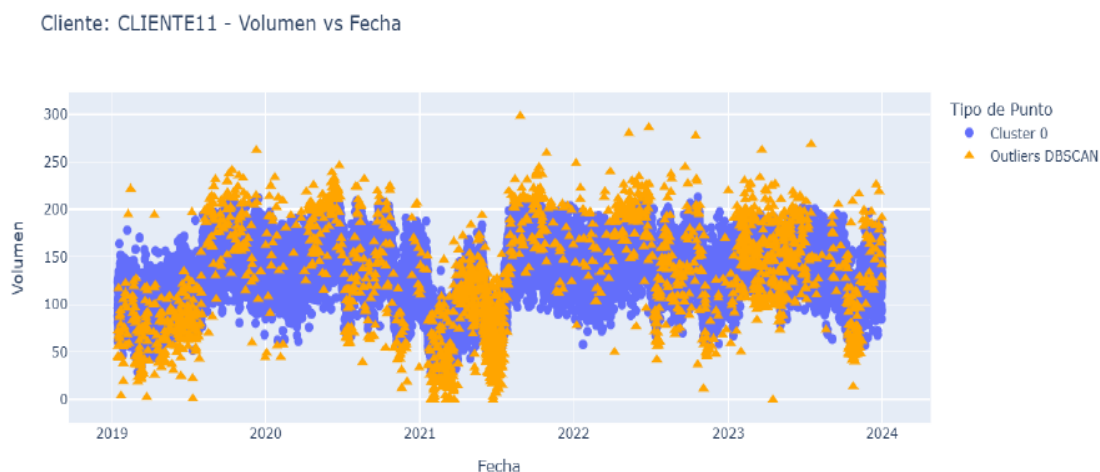
Se implementó el algoritmo DBSCAN considerando únicamente tres variables: presión, temperatura y volumen. El modelo fue ejecutado cliente por cliente, con selección individual del parámetro eps mediante un Knee-Graph y un valor fijo de min_samples = 6.

Hallazgos clave:

- El número promedio de clústeres por cliente fue bajo (5), con generalización excesiva del consumo.
- La mayoría de outliers aparecían superpuestos en zonas de alta densidad, dificultando la identificación de fugas.
- No se lograron clústeres representativos de comportamientos atípicos persistentes.

A continuación, el resultado obtenido sobre un cliente para el primer modelo:

Gráfico 7: Resultados Modelo 1 – Cliente 11



2.2. DBSCAN con 7 Features

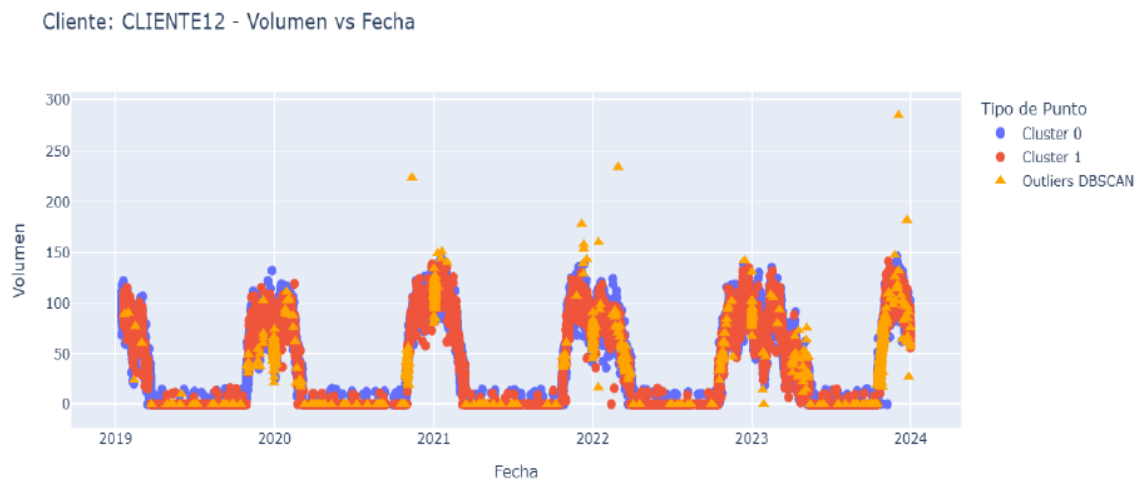
Se ampliaron las variables de entrada añadiendo componentes temporales: mes, día de la semana, semana del año y fin de semana, manteniendo DBSCAN como algoritmo base. Se incrementó min_samples a 14 por el aumento de dimensiones.

Resultados:

- El promedio de clústeres por cliente aumentó levemente (5.8).
- Se observó una mejor segmentación temporal, identificando posibles anomalías recurrentes.
- Persistieron los falsos positivos debido a ruido en la señal de consumo.

A continuación, el resultado obtenido sobre un cliente para el segundo modelo:

Gráfico 10: Resultados Modelo 2 – Cliente 12



2.3. DBSCAN sobre errores de SVR Simple

Se entrenó un modelo de regresión SVR para predecir el volumen esperado. Luego, DBSCAN fue aplicado sobre los errores residuales para detectar desviaciones significativas.

Ventajas observadas:

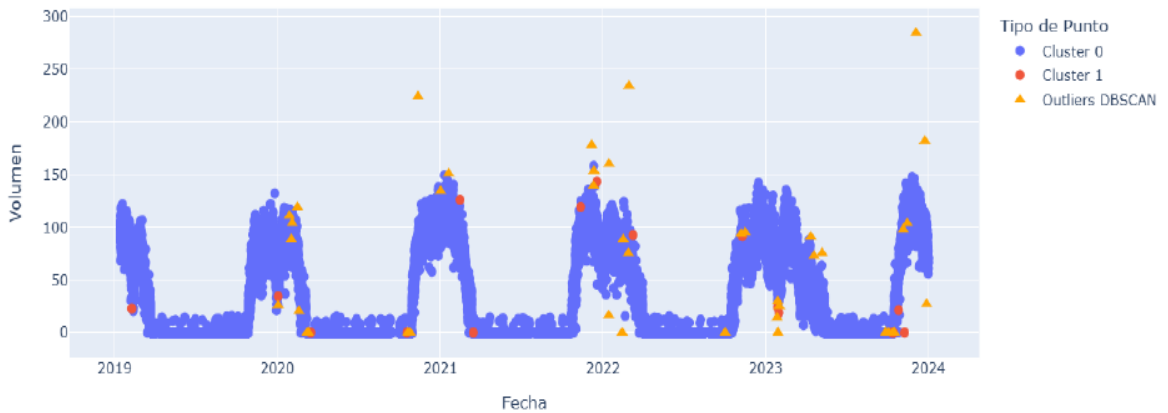
- Reducción considerable del ruido en la detección de outliers.
- Detección efectiva de puntos aislados anómalos.

- Limitación: no se identificaron patrones anómalos persistentes o filamentos de outliers.

A continuación, el resultado obtenido sobre un cliente para el tercer modelo:

Gráfico 14: Resultados Modelo 3 – Cliente 12

Cliente: CLIENTE12 - Volumen vs Fecha



2.4. DBSCAN con SVR Complejo

Se combinó SVR y DBSCAN en un pipeline iterativo más robusto:

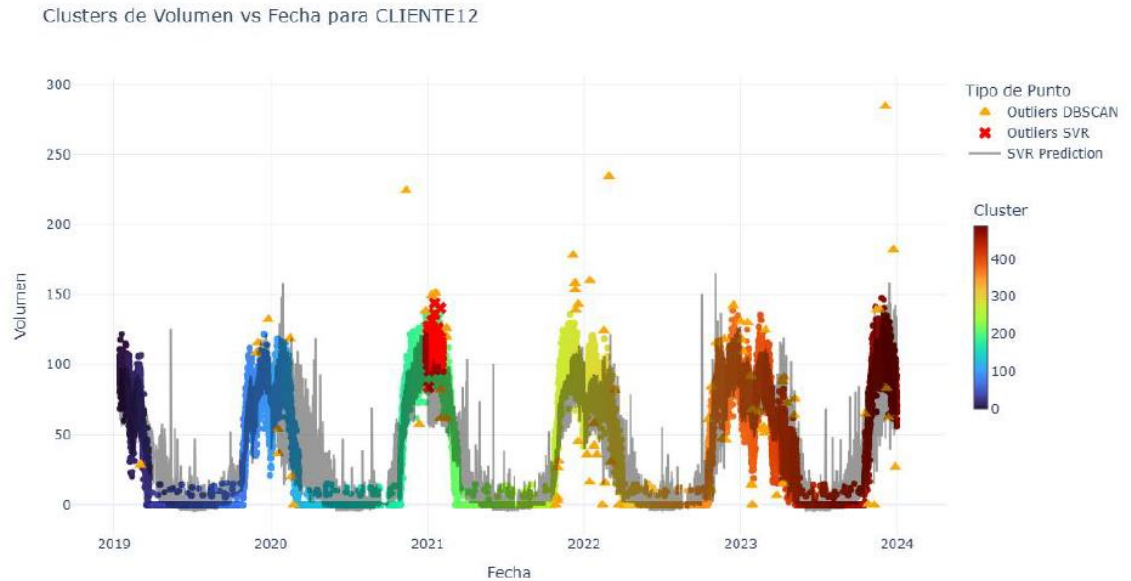
1. Entrenamiento inicial de SVR tras eliminar outliers extremos.
2. Identificación de clústeres mediante DBSCAN con componentes temporales y de consumo.
3. Reentrenamiento iterativo del SVR excluyendo clústeres con errores residuales altos hasta estabilizarse.

Resultados destacados:

- Mayor número de clústeres (promedio de 1.396), reflejando una segmentación granular y detallada.
- Identificación efectiva de filamentos de outliers en series temporales.
- Mejora sustancial en la detección de patrones anómalos continuos (potenciales fugas).

A continuación, el resultado del modelo final sobre un cliente

Gráfico 18: Resultados Modelo 4 – Cliente 12



Para mayor detalle consultar el anexo 2. “Reporte de Selección y Parametrización de Modelos”

2.5. Comparación de Desempeño

Para evaluar los modelos desarrollados, se utilizaron tres métricas estándar en problemas de clustering: **Silhouette Score**, **Índice de Davies-Bouldin** e **Índice de Calinski-Harabasz**. La tabla siguiente resume los promedios obtenidos por cada modelo:

Métrica	Modelo 1	Modelo 2	Modelo 3	Modelo 4
Silhouette Score (↑ mejor)	0.228	0.197	0.822	0.183
Davies-Bouldin Index (↓ mejor)	0.783	1.627	0.269	16.047
Calinski-Harabasz Score (↑ mejor)	4,977	6,420	36,713	1,518

Interpretación:

- El **Modelo 3** obtuvo las mejores métricas promedio, indicando una buena cohesión y separación entre clústeres. Sin embargo, su limitación fue la incapacidad para detectar filamentos de anomalías temporales.
- El **Modelo 4**, aunque penalizado por las métricas debido al elevado número de microclústeres, fue considerado el más alineado con los objetivos del proyecto: detectar patrones específicos de consumo anómalo asociados que son críticos para la detección de fugas de gas.

3. Rúbrica de evaluación diligenciada:

A continuación, la rúbrica diligenciada:

Aspecto	Requerimiento	¿Cumplido ?	Justificación
Requerimientos analítica	Disminuir el número de eventos críticos no detectados en el servicio	Parcialmente	Modelo SVR+DBSCAN implementado. Falta validar tasa de cobertura $\geq 90\%$ con históricos o eventos reales.
Requerimientos analítica	Incrementar la eficiencia operativa del sistema de distribución	Pendiente	No se ha realizado aún la métrica comparativa antes/después. Requiere evaluación en entorno operativo real.
Requerimientos analítica	Mejorar el control operativo mediante segmentación de consumos atípicos	Cumplido	Se evaluó Silhouette, Davies-Bouldin y precisión por clúster. Modelo 4 permite agrupación efectiva de comportamientos.
Requerimientos analítica	Mejorar la satisfacción del cliente mediante monitoreo y alertas tempranas	Pendiente	Dashboard implementado, pero aún no se ha aplicado una encuesta de satisfacción ni pruebas formales de usabilidad.
Requerimientos de desempeño	Segmentar patrones para detectar comportamientos irregulares	Cumplido	Modelos evaluados con Silhouette, Davies-Bouldin y Calinski-Harabasz. DBSCAN ajustado con minPts y eps calibrados.
Requerimiento funcional	Visualización funcional de resultados y comparaciones	Cumplido	Dash muestra resultados correctamente. Coincidencia visual con detección $\geq 95\%$. Dashboard interactivo validado visualmente.
Requerimiento funcional	Estabilidad operativa del sistema en Railway	Cumplido	Sistema desplegado en Railway. Uptime registrado $> 99\%$ durante pruebas de carga simuladas.

Requerimiento funcional	Tiempo de respuesta del sistema ante nuevos registros	Parcialmente	Procesamiento de datos actuales tiempo de respuesta para consultar datos 5 segundos máximo y aumentando dependiendo la cantidad de años filtrados, pendiente realizar pruebas para datos nuevos se estima que el proceso tarde máximo 10 minutos
Requerimiento funcional	Usabilidad básica del tablero interactivo	Inicialmente	Filtros y navegación funcionales (<2s). Se recomienda recolectar feedback tras uso por usuario final.

4. Archivos de código desarrollados para la implementación del prototipo y las pruebas correspondientes (repositorio de GitHub):

El desarrollo del prototipo funcional y la ejecución de pruebas del sistema de detección de anomalías para clientes de Contugas se gestionó mediante un repositorio GitHub de control de versiones:

https://github.com/SOFIASALAZARSUAZA/PROYECTO_CONTUGAS_REV1

4.1. Estructura del Repositorio

El repositorio está organizado de forma modular para separar los componentes del sistema:

PROYECTO_CONTUGAS_REV1/

```

├── assets/           # Archivos de soporte visual
├── backend/         # Lógica del sistema y API
│   ├── app.py       # Script principal del servidor Flask
│   ├── admin_modelos.py # Control de lógica y parametrización de modelos
│   ├── entrenamiento_modelo.py # Entrenamiento de modelos de análisis
│   ├── modelo.py    # Definición de SVR + DBSCAN final
│   ├── reentrenar_modelo.bat # Script batch para ejecución automática
│   └── data/        # Datos de prueba y simulación

```



```

|   ├── input/           # Archivos fuente para carga
|   └── frontend/        # Interfaz HTML/JS/CSS
|       ├── index.html
|       ├── script.js
|       └── style.css
└── requirements.txt     # Dependencias necesarias
└── railway.json         # Configuración para despliegue en Railway
└── README.md           # Instrucciones de instalación y uso

```

4.2. Descripción Funcional de Componentes

Script	Función principal	Entradas	Salidas	Comentarios
<code>app.py</code>	Levanta el servidor Flask y expone endpoints REST.	Ninguna directa; usa funciones internas.	HTML + JSON vía HTTP.	Interactúa con el frontend y llama los modelos según las rutas.
<code>modelo.py</code>	Define la lógica completa del modelo 4 (SVR iterativo + DBSCAN).	DataFrame con volumen, presión, temperatura, fecha.	DataFrame con etiquetas de outliers y clústeres.	Es el núcleo del sistema de análisis; contiene la lógica del pipeline explicado en el informe.
<code>entrenamiento_modelo.py</code>	Entrena modelos alternativos para comparación (modelos 1 a 3).	Datos de consumo de un cliente.	Modelos serializados o DataFrame etiquetado.	Ideal para experimentos y pruebas.
<code>admin_modelos.py</code>	Orquesta la ejecución y selección de modelos.	Nombre del modelo + parámetros.	Resultados del modelo en formato estándar.	Facilita pruebas A/B entre enfoques.
<code>script.js</code>	Controla interacciones en la interfaz web.	Eventos del usuario (click, cambio de cliente, etc.).	Actualiza DOM con KPIs y gráficos.	Comunicación por fetch con <code>app.py</code> .

reentrenar_modelo.bat	Automatiza el reentrenamiento local.	Ninguna.	Llama a entrenamiento_modelo.py.	Puede ser adaptado para tareas cron.
-----------------------	--------------------------------------	----------	----------------------------------	--------------------------------------

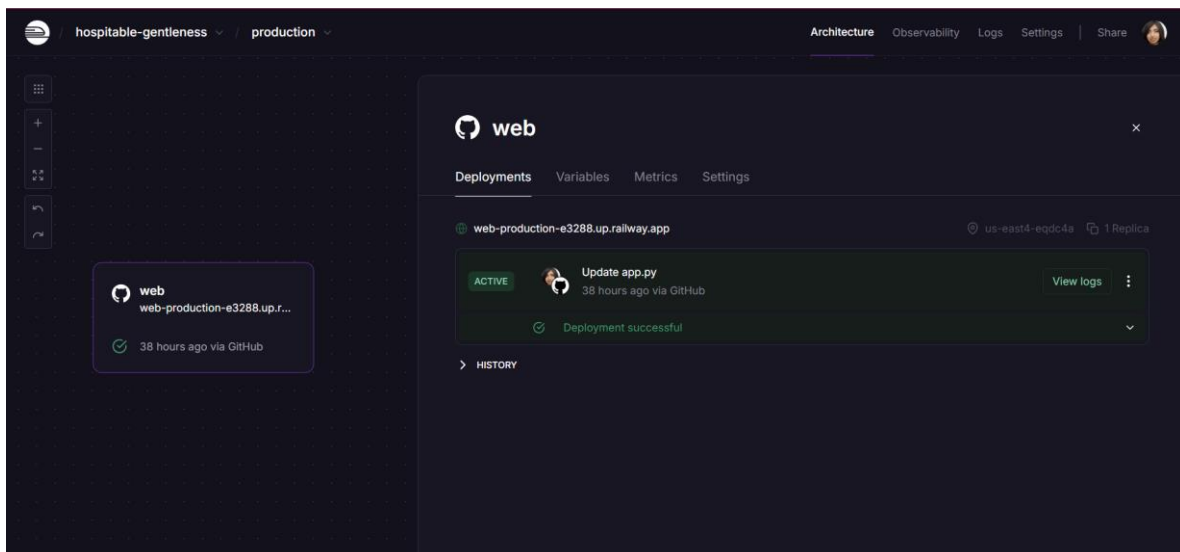
4.3. Despliegue RailWay

Con el objetivo de facilitar la demostración del prototipo funcional, el sistema fue desplegado utilizando la plataforma [Railway](#). Esta solución permite la publicación de aplicaciones web de forma sencilla, con integración directa desde GitHub.

4.3.1. Configuración del despliegue

- El repositorio fue enlazado directamente desde Railway mediante autenticación GitHub.
- El script principal (backend/app.py) fue establecido como punto de entrada para ejecutar la API con Flask.
- Railway detectó automáticamente las dependencias desde requirements.txt.
- El servicio fue desplegado en la región us-east4-eqdc4a, y se encuentra actualmente **activo** y accesible para pruebas.

A continuación, se presenta la evidencia del despliegue exitoso en el entorno de producción:



URL pública del sistema:
<https://web-production-e3288.up.railway.app>

